pfi

# Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

# Classes: A Bond Object

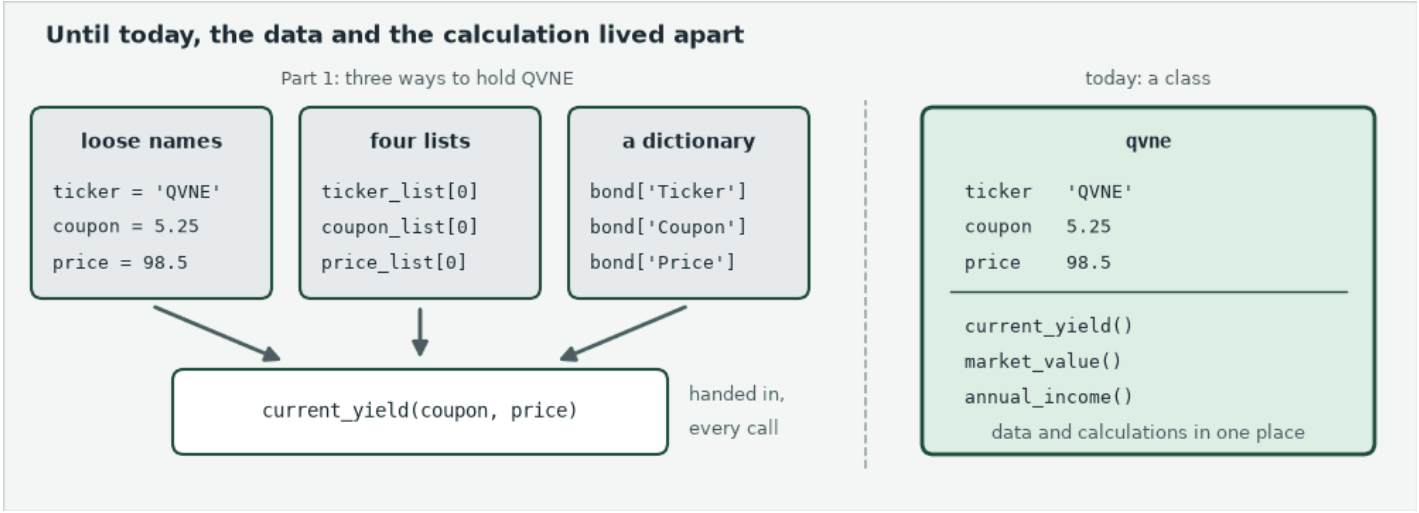
WEEK 1 · DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# The desk lead's sixth note

*Every day we hand current_yield a coupon and a price, and on Day 3 a price of zero walked straight in. Build me a Bond that carries its own terms, works out its own numbers, refuses a bad price, and prints in words. Then tie the book's value and income from four Bonds.*




**A class defines our own kind of object, one that holds data and calculations together.**

# 4.1 Every value has a class, so write one

### Python's own classes

```
coupon = 5.25
ticker = 'QVNE'
print(type(coupon))
print(type(ticker))
print(ticker.lower())
<class 'float'>
<class 'str'>
qvne
```

A class sets out two things: the data each object carries, and the methods it can run.

### 4.2 A first class: a blank form

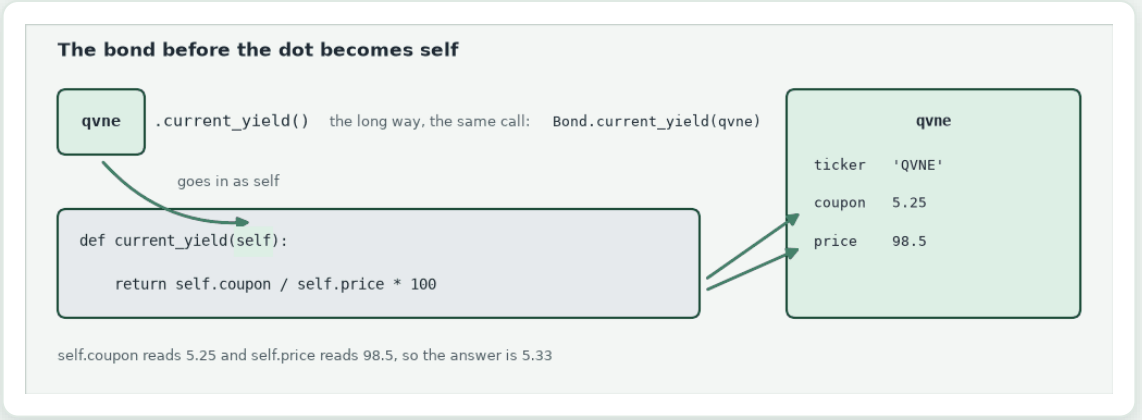
```
class Bond:
    def set_terms(self, ticker, coupon, price):
        self.ticker = ticker
        self.coupon = coupon
        self.price = price
    def current_yield(self):
        return self.coupon / self.price * 100
```

In self.ticker = ticker, the value handed in, on the right, is stored on the object, on the left.

**In VBA terms, a class is Insert, Class Module, and self is Me.**

# Make one bond, then ask it

```
qvne = Bond()
qvne.set_terms('QVNE', 5.25, 98.5)
print(type(qvne))
<class '__main__.Bond'>
print(qvne.coupon)
print(qvne.current_yield())
5.25
5.32994923857868
```




| You write | Brackets | What it is |
|---|---|---|
| `qvne.coupon` | none | an attribute: data stored on the bond |
| `qvne.current_yield()` | yes | a method: a calculation the bond runs |

**Inside the class self is this bond; outside, use the bond's own name.**

# 4.3 One class, many objects, nothing alike yet

### A second bond

```
dnmr = Bond()
dnmr.set_terms('DNMR', 6.125, 103.25)
print(dnmr.current_yield())
print(qvne.current_yield())
5.932203389830509
5.32994923857868
```

### A sector on one bond only

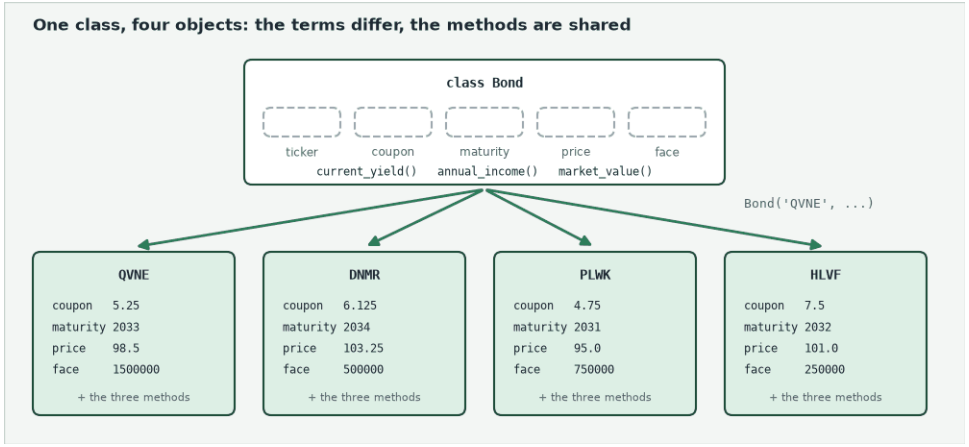
```
qvne.sector = 'Energy'
print(qvne.sector)
Energy
print(dnmr.sector)
AttributeError: 'Bond' object has no attribute 'sector'
```

### A bond never filled in

```
plwk = Bond()
print(plwk.coupon)
AttributeError: 'Bond' object has no attribute 'coupon'
```

**Every Bond needs the same terms from the moment it is made.**

# 4.4 The initializer: made and filled in one step




## The new block: __init__

```
def __init__(self, ticker, coupon, maturity, price, face):
    self.ticker = ticker
    self.coupon = coupon
    self.maturity = maturity
    self.price = price
    self.face = face
```

Below it: current_yield(), annual_income(), market_value()

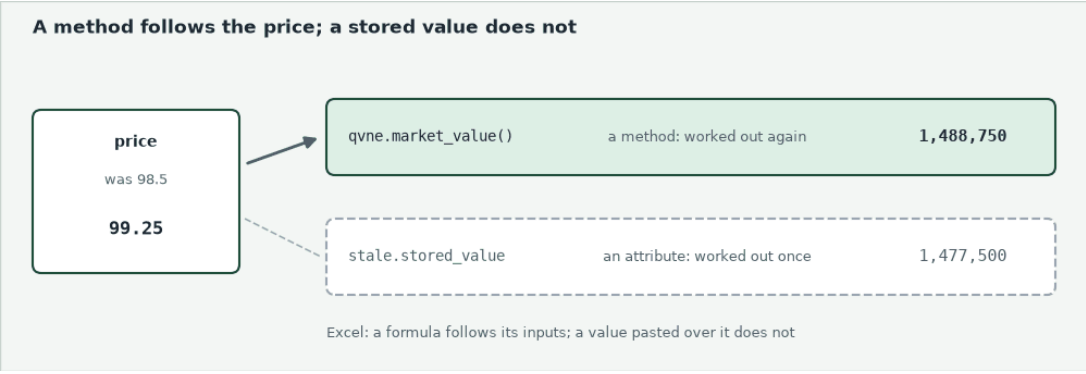
```
qvne = Bond('QVNE', 5.25, 2033, 98.5, 1500000)
dnmr = Bond('DNMR', 6.125, 2034, 103.25, 500000)
plwk = Bond('PLWK', 4.75, 2031, 95.0, 750000)
hlvf = Bond('HLVF', 7.5, 2032, 101.0, 250000)
print(plwk.coupon)
4.75
```

```
osmc = Bond('OSMC', 5.0, 2035, 100.0)
TypeError: Bond.__init__() missing 1 required positional argument: 'face'
```

Excel side by side. Type =ROUND(98.5) and Excel says you have entered too few arguments, and will not take the formula.

# 4.5 Value, income, and a stored value that goes stale

```
print(f'Market value: ${qvne.market_value():,.2f}')
print(f'Annual coupon income: ${qvne.annual_income():,.2f}')
Market value: $1,477,500.00
Annual coupon income: $78,750.00
qvne.price = 99.25
print(f'Market value: ${qvne.market_value():,.2f}')
Market value: $1,488,750.00
```




## Worked out once, when the bond is made

```
stale = BondWithStoredValue('QVNE', 98.5, 1500000)
print(f'Before the mark: ${stale.stored_value:,.2f}')
stale.price = 99.25
print(f'After the mark:  ${stale.stored_value:,.2f}')
Before the mark: $1,477,500.00
After the mark:  $1,477,500.00
```

## Excel side by side

A value pasted over a formula looks the same, and stops following the price.

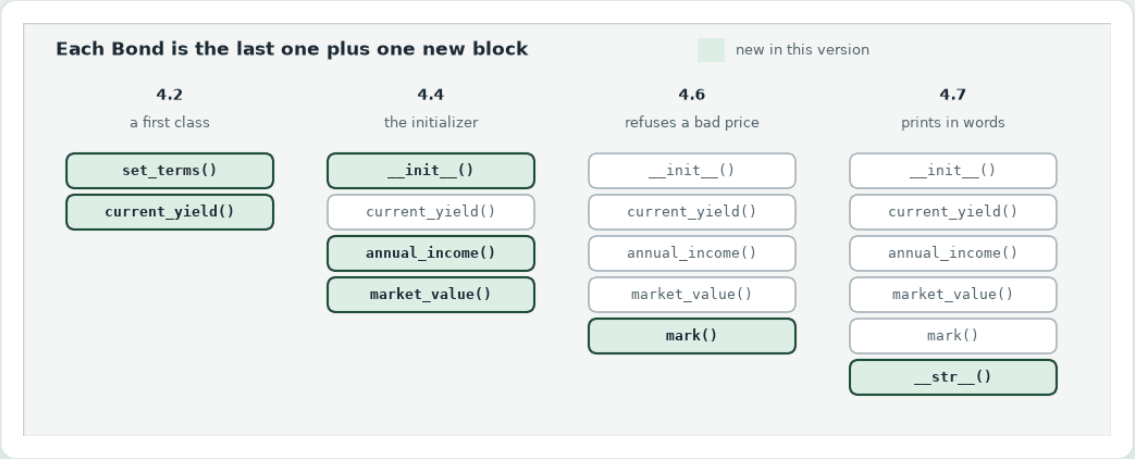
**Store the facts as attributes; calculate anything that depends on them in a method.**

# 4.6 A bond that refuses a bad price

## A price of zero walks in

```
qvne.price = 0
print(qvne.current_yield())
ZeroDivisionError: float division by zero
```

The error points at the arithmetic, not at the bad price: Day 3's ZeroDivisionError.




## The new block, added at the bottom of 4.4's class

```
    def mark(self, new_price):
        if new_price <= 0:
            raise ValueError(f'the price must be above zero, and it is {new_price}')
        self.price = new_price
```

```
qvne.mark(0)
ValueError: the price must be above zero, and it is 0
print(qvne.price)
99.25
```

**The Bond refuses a bad price itself: Day 3's raise, now inside the class.**

# 4.7 Printing an object

## What print shows

```
print(qvne)
<__main__.Bond object at 0x...>
```

0x... is where it sits in memory: a new address on each run

| Method | Python calls it when |
|---|---|
| `__init__` | a Bond is made |
| `__str__` | a Bond is printed |
| `__repr__` | a list of Bonds is printed; not written today |

## The new block: __str__ gives it words

```
    def __str__(self):
        terms = f'{self.ticker} {self.coupon}% of {self.maturity}'
        return f'{terms}, current yield {self.current_yield():.2f}%'
print(qvne)
QVNE 5.25% of 2033, current yield 5.33%
```

### Excel side by side

__str__ builds a label from the facts, as a formula does: with ticker, coupon, and maturity in A2:C2, =A2&" "&B2&"% of "&C2 returns QVNE 5.25% of 2033.

**Now a Bond prints in words.**

# 4.8 Four bonds in a list

**A list of Bonds reads like a holdings sheet**

attributes: plain columns | methods: formula columns

the class →

| Ticker | Coupon | Maturity | Price | Face | Current yield | Market value | Income |
|---|---|---|---|---|---|---|---|
| QVNE | 5.25 | 2033 | 98.5 | 1,500,000 | 5.33 | 1,477,500 | 78,750 |
| DNMR | 6.125 | 2034 | 103.25 | 500,000 | 5.93 | 516,250 | 30,625 |
| PLWK | 4.75 | 2031 | 95.0 | 750,000 | 5.00 | 712,500 | 35,625 |
| HLVF | 7.5 | 2032 | 101.0 | 250,000 | 7.43 | 252,500 | 18,750 |

each row: one object

fill down = the loop

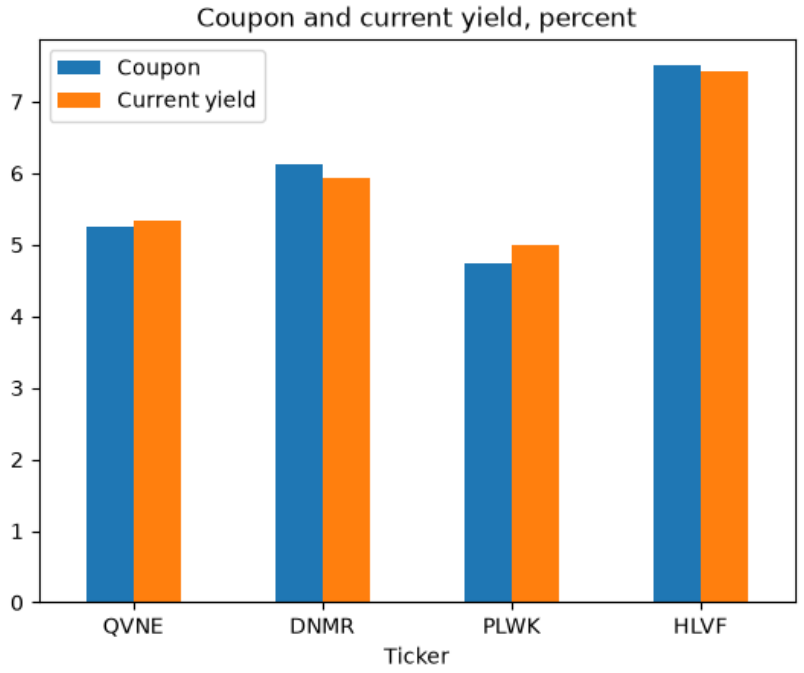
```
portfolio = [qvne, dnmr, plwk, hlvf]
for bond in portfolio:
    print(bond)
QVNE 5.25% of 2033, current yield 5.33%
DNMR 6.125% of 2034, current yield 5.93%
PLWK 4.75% of 2031, current yield 5.00%
HLVF 7.5% of 2032, current yield 7.43%
```

### Excel side by side

In an Excel Table (Insert, Table), a new row picks up the formula columns by itself, as a new Bond comes with its methods.

**Each bond carries everything it needs.**

# Discount bonds and their yield




```
discount_tickers = [bond.ticker for bond in portfolio if bond.price < 100]
print(discount_tickers)
['QVNE', 'PLWK']
```

### Excel side by side

With tickers in A2:A5 and prices in D2:D5, =FILTER(A2:A5,D2:D5<100) returns the same two; FILTER needs Excel 2021 or later, or Microsoft 365.

**Below par, the current yield is above the coupon; above par, below it.**

# Rank by value, pick the top coupon, tie the book

```
ranked = sorted(portfolio, key=lambda bond: bond.market_value(), reverse=True)
for bond in ranked:
    print(f'{bond.ticker}: ${bond.market_value():,.2f}')
QVNE: $1,477,500.00
PLWK: $712,500.00
DNMR: $516,250.00
HLVF: $252,500.00
```

```
highest = max(portfolio, key=lambda bond: bond.coupon)
print(highest)
HLVF 7.5% of 2032, current yield 7.43%
```

### Excel side by side

With tickers in A2:A5 and coupons in B2:B5, =INDEX(A2:A5,MATCH(MAX(B2:B5),B2:B5,0)) returns the top coupon's ticker: MAX finds the number, MATCH its row, INDEX the ticker on that row.

```
assert math.isclose(book_value, 2958750), 'value does not tie'
assert math.isclose(book_income, 163750), 'income does not tie'
print(f'Book ties: ${book_value:,.2f} value, ${book_income:,.2f} income')
Book ties: $2,958,750.00 value, $163,750.00 income
```

# The note, answered

| The note asked | Answer |
|---|---|
| Its own terms and numbers | current_yield(), market_value(), annual_income(), nothing handed in |
| Refuse a bad price | qvne.mark(0) stops with a ValueError that says why |
| Print in words | QVNE 5.25% of 2033, current yield 5.33% |
| The book from four Bonds | two assert lines tie the value and the income |

**On an object, a dot now reads plainly: no brackets, data; brackets, a calculation.**

**On Day 5 a desk takes over six bonds typed into an email, and the case study puts the whole week to work, from loose values to one pandas table.**